

Linked List Data Structure in Java

A **Linked List** is a linear data structure where elements are stored in nodes, and each node contains a reference (or pointer) to the next node in the sequence. This allows for dynamic memory allocation, making it easy to insert and delete elements without reorganizing the entire structure.

Types of Linked Lists

1. **Singly Linked List:** Each node points to the next node, allowing traversal in one direction.
2. **Doubly Linked List:** Each node points to both the next and the previous nodes, allowing traversal in both directions.
3. **Circular Linked List:** The last node points back to the first node, forming a circle.
4. **Circular Doubly Linked List:** Combines features of both circular and doubly linked lists.

Implementation in Java

1. Singly Linked List

Node Class:

```
java
VerifyOpen In EditorRunCopy code
1public class Node {
2    int data;
3    Node next;
4
5    public Node(int data) {
6        this.data = data;
7        this.next = null;
8    }
9}
```

Singly Linked List Class:

```
java
VerifyOpen In EditorRunCopy code
1public class SinglyLinkedList {
2    Node head;
3
4    // Insert at the end
5    public void insert(int data) {
6        Node newNode = new Node(data);
7        if (head == null) {
8            head = newNode;
9        } else {
10           Node current = head;
11           while (current.next != null) {
12               current = current.next;
13           }
14           current.next = newNode;
15        }
16    }
17
18    // Display the list
19    public void display() {
20        Node current = head;
21        while (current != null) {
22            System.out.print(current.data + " -> ");
23            current = current.next;
24        }
25    }
26}
```

```

24     }
25     System.out.println("null");
26 }
27
28 public static void main(String[] args) {
29     SinglyLinkedList list = new SinglyLinkedList();
30     list.insert(10);
31     list.insert(20);
32     list.insert(30);
33     list.display(); // Output: 10 -> 20 -> 30 -> null
34 }
35}

```

2. Doubly Linked List

Node Class:

```

java
VerifyOpen In EditorRunCopy code
1public class DoublyNode {
2    int data;
3    DoublyNode next;
4    DoublyNode previous;
5
6    public DoublyNode(int data) {
7        this.data = data;
8        this.next = null;
9        this.previous = null;
10    }
11}

```

Doubly Linked List Class:

```

java
VerifyOpen In EditorRunCopy code
1public class DoublyLinkedList {
2    DoublyNode head;
3
4    // Insert at the end
5    public void insert(int data) {
6        DoublyNode newNode = new DoublyNode(data);
7        if (head == null) {
8            head = newNode;
9        } else {
10            DoublyNode current = head;
11            while (current.next != null) {
12                current = current.next;
13            }
14            current.next = newNode;
15            newNode.previous = current;
16        }
17    }
18
19    // Display the list
20    public void display() {
21        DoublyNode current = head;
22        while (current != null) {
23            System.out.print(current.data + " <-> ");
24            current = current.next;
25        }
26        System.out.println("null");
27    }
28}

```

```

29     public static void main(String[] args) {
30         DoublyLinkedList list = new DoublyLinkedList();
31         list.insert(10);
32         list.insert(20);
33         list.insert(30);
34         list.display(); // Output: 10 <-> 20 <-> 30 <-> null
35     }
36}

```

3. Circular Linked List

Node Class:

```

java
VerifyOpen In EditorRunCopy code
1public class CircularNode {
2    int data;
3    CircularNode next;
4
5    public CircularNode(int data) {
6        this.data = data;
7        this.next = null;
8    }
9}

```

Circular Linked List Class:

```

java
VerifyOpen In EditorRunCopy code
1public class CircularLinkedList {
2    CircularNode head;
3
4    // Insert at the end
5    public void insert(int data) {
6        CircularNode newNode = new CircularNode(data);
7        if (head == null) {
8            head = newNode;
9            newNode.next = head; // Point to itself
10       } else {
11           CircularNode current = head;
12           while (current.next != head) {
13               current = current.next;
14           }
15           current.next = newNode;
16           newNode.next = head; // Make it circular
17       }
18   }
19
20   // Display the list
21   public void display() {
22       if (head == null) return;
23       CircularNode current = head;
24       do {
25           System.out.print(current.data + " -> ");
26           current = current.next;
27       } while (current != head);
28       System.out.println("(back to head)");
29   }
30
31   public static void main(String[] args) {
32       CircularLinkedList list = new CircularLinkedList();
33       list.insert(10);
34       list.insert(20);

```

```
35         list.insert(30);
36         list.display(); // Output: 10 -> 20 -> 30 -> (back to head)
37     }
38 }
```

Advantages of Linked Lists

- **Dynamic Size:** Can grow and shrink as needed.
- **Efficient Insertions/Deletions:** Adding or removing elements is generally faster than in arrays.
- **No Wasted Space:** Memory is allocated as needed.

Disadvantages of Linked Lists

- **Memory Overhead:** Requires extra memory for pointers.
- **No Random Access:** Must traverse the list to access elements.
- **Cache Performance:** Poorer cache performance compared to arrays.

Conclusion

Linked lists are a fundamental data structure in Java, providing flexibility and efficiency for dynamic data management. Understanding how to implement and manipulate linked lists is crucial for software development and algorithm design.

Applications of Linked Lists in Java

Linked lists are versatile data structures that have numerous applications in programming and software development. Here are some key applications and advantages of using linked lists in Java:

1. Dynamic Memory Allocation

- **Description:** Linked lists allow for dynamic memory allocation, meaning that memory can be allocated as needed during runtime. This is particularly useful when the number of elements is not known in advance.
- **Advantage:** Unlike arrays, which require a fixed size, linked lists can grow and shrink as needed without wasting memory.

2. Implementation of Other Data Structures

- **Stacks and Queues:** Linked lists are often used to implement stacks and queues due to their efficient insertion and deletion operations.
- **Graphs:** Adjacency lists for graph representation can be implemented using linked lists, allowing for efficient traversal and manipulation of graph data.

3. Real-World Applications

- **Music Players:** Songs can be linked in a playlist, allowing users to navigate to the next or previous song easily.
- **Web Browsers:** The history of visited pages can be managed using a doubly linked list, enabling users to go back and forth between pages.

- **Image Viewers:** Images can be linked to allow users to navigate through a gallery using next and previous buttons.

4. Efficient Insertions and Deletions

- **Description:** Linked lists provide $O(1)$ time complexity for insertions and deletions at any position, as only pointers need to be updated.
- **Advantage:** This is significantly faster than arrays, where such operations can take $O(n)$ time due to the need for shifting elements.

5. Memory Management in Operating Systems

- **Description:** Linked lists can be used for memory management, such as keeping track of free memory blocks.
- **Example:** In operating systems, linked lists can manage processes and their scheduling (e.g., round-robin scheduling using circular linked lists).

6. Manipulation of Polynomials

- **Description:** Linked lists can represent polynomials where each node contains a coefficient and an exponent, allowing for efficient polynomial operations.

7. LRU Cache Implementation

- **Description:** A Least Recently Used (LRU) cache can be implemented using a doubly linked list to keep track of the most recently accessed items, allowing for quick updates and removals.

Example Code: Basic Linked List Implementation in Java

Here's a simple implementation of a singly linked list in Java, demonstrating insertion and traversal:

```

1 java
2 VerifyOpen In EditorRunCopy code
3 class LinkedList {
4     // Node class representing each element in the linked list
5     class Node {
6         int data;
7         Node next;
8
9         Node(int data) {
10             this.data = data;
11             this.next = null;
12         }
13     }
14
15     private Node head; // Head of the list
16
17     // Method to insert a new node at the end of the list
18     public void insertAtEnd(int data) {
19         Node newNode = new Node(data);
20         if (head == null) {
21             head = newNode; // If the list is empty, set head to new node
22         } else {
23             Node temp = head;
24             while (temp.next != null) {
25                 temp = temp.next; // Traverse to the last node

```

```

24         }
25         temp.next = newNode; // Link the new node
26     }
27 }
28
29 // Method to traverse and print the linked list
30 public void traverse() {
31     Node temp = head;
32     while (temp != null) {
33         System.out.print(temp.data + " -> ");
34         temp = temp.next;
35     }
36     System.out.println("null");
37 }
38
39 public static void main(String[] args) {
40     LinkedList list = new LinkedList();
41     list.insertAtEnd(10);
42     list.insertAtEnd(20);
43     list.insertAtEnd(30);
44     list.traverse(); // Output: 10 -> 20 -> 30 -> null
45 }
46}

```

Conclusion

Linked lists are a powerful data structure with a wide range of applications in Java programming. Their dynamic nature and efficient operations make them suitable for various scenarios, from implementing complex data structures to managing real-world applications like music players and web browsers.

Linked List vs Array Comparison in Java

When comparing **Linked Lists** and **Arrays** in Java, it's essential to understand their fundamental differences, advantages, and disadvantages. Below is a detailed comparison based on various parameters.

1. Memory Allocation

- **Array:**
 - Stores elements in **contiguous memory locations**.
 - Memory allocation is done at **compile time**.
 - Fixed size; cannot be resized after creation.
- **Linked List:**
 - Stores elements in **non-contiguous memory locations**.
 - Memory allocation is done at **runtime**.
 - Dynamic size; can grow or shrink as needed.

2. Structure

- **Array:**
 - Consists of a single data field for each element.
 - Elements are independent of each other.
- **Linked List:**

- Composed of nodes, each containing:
 - A **data field**.
 - A **reference (pointer)** to the next node (and possibly the previous node in a doubly linked list).

3. Access Time

- **Array:**
 - **Accessing an element:** $O(1)$ (constant time) due to direct indexing.
- **Linked List:**
 - **Accessing an element:** $O(n)$ (linear time) since it requires traversal from the head node.

4. Insertion and Deletion

- **Array:**
 - **Insertion/Deletion:** $O(n)$ (linear time) because elements may need to be shifted.
- **Linked List:**
 - **Insertion/Deletion:** $O(1)$ (constant time) if the position is known, as it only involves changing pointers.

5. Memory Efficiency

- **Array:**
 - More memory-efficient when the size is known in advance.
 - Less overhead since it only stores the data.
- **Linked List:**
 - Requires more memory per element due to storage of pointers.
 - More efficient when the number of elements is unknown or varies significantly.

6. Locality of Reference

- **Array:**
 - Better cache performance due to contiguous memory allocation, leading to faster access times for nearby elements.
- **Linked List:**
 - Poorer cache performance as nodes can be scattered throughout memory.

7. Use Cases

- **Array:**
 - Best when:
 - The number of elements is known beforehand.
 - Fast access to elements is required.
 - Memory overhead needs to be minimized.
- **Linked List:**
 - Best when:
 - Frequent insertions and deletions are expected.
 - The number of elements is not known in advance.

Conclusion

Choosing between an array and a linked list depends on the specific requirements of your application. If you need fast access and know the size of your data, arrays are preferable. If you need flexibility in size and frequent modifications, linked lists are the better choice.

Example Code in Java

Here's a simple implementation of both an array and a linked list in Java:

Array Example

```
java
VerifyOpen In EditorRunCopy code
1public class ArrayExample {
2    public static void main(String[] args) {
3        int[] array = new int[5]; // Declare an array of size 5
4        array[0] = 10; // Inserting elements
5        array[1] = 20;
6
7        // Accessing elements
8        System.out.println("Element at index 1: " + array[1]); // Output: 20
9
10       // Inserting an element (requires shifting)
11       for (int i = array.length - 1; i > 0; i--) {
12           array[i] = array[i - 1]; // Shift elements
13       }
14       array[0] = 5; // Insert at the beginning
15   }
16}
```

Linked List Example

```
java
VerifyOpen In EditorRunCopy code
1class Node {
2    int data;
3    Node next;
4
5    Node(int data) {
6        this.data = data;
7        this.next = null;
8    }
9}
10
11public class LinkedListExample {
12    Node head; // Head of the list
13
14    // Method to insert a new node at the beginning
15    public void insertAtBeginning(int data) {
16        Node newNode = new Node(data);
17        newNode.next = head;
18        head = newNode;
19    }
20
21    // Method to print the linked list
22    public void printList() {
23        Node current = head;
24        while (current != null) {
25            System.out.print(current.data + " ");
26            current = current.next;
27        }
28    }
29}
```



```

28     }
29
30     public static void main(String[] args) {
31         LinkedListExample list = new LinkedListExample();
32         list.insertAtBeginning(10);
33         list.insertAtBeginning(20);
34         list.printList(); // Output: 20 10
35     }
36}

```

Final Thoughts

Both data structures have their unique strengths and weaknesses. Understanding these will help you make informed decisions when designing your applications. If you have further questions or need more examples, feel free to ask!

Final Answer

Linked List Data Structure in Java

A **Linked List** is a linear data structure where elements are stored in nodes, and each node contains a reference (or pointer) to the next node in the sequence. This allows for dynamic memory allocation, making it easy to insert and delete elements without reorganizing the entire structure.

Types of Linked Lists

1. **Singly Linked List:** Each node points to the next node, allowing traversal in one direction.
2. **Doubly Linked List:** Each node points to both the next and the previous nodes, allowing traversal in both directions.
3. **Circular Linked List:** The last node points back to the first node, forming a circle.
4. **Circular Doubly Linked List:** Combines features of both circular and doubly linked lists.

Implementation in Java

1. Singly Linked List

Node Class:

```

java
VerifyOpen In EditorRunCopy code
1public class Node {
2    int data;
3    Node next;
4
5    public Node(int data) {
6        this.data = data;
7        this.next = null;
8    }
9}

```

Singly Linked List Class:

```

java
VerifyOpen In EditorRunCopy code
1public class SinglyLinkedList {
2    Node head;

```

```

3
4 // Insert at the end
5 public void insert(int data) {
6     Node newNode = new Node(data);
7     if (head == null) {
8         head = newNode;
9     } else {
10        Node current = head;
11        while (current.next != null) {
12            current = current.next;
13        }
14        current.next = newNode;
15    }
16 }
17
18 // Display the list
19 public void display() {
20     Node current = head;
21     while (current != null) {
22         System.out.print(current.data + " -> ");
23         current = current.next;
24     }
25     System.out.println("null");
26 }
27
28 public static void main(String[] args) {
29     SinglyLinkedList list = new SinglyLinkedList();
30     list.insert(10);
31     list.insert(20);
32     list.insert(30);
33     list.display(); // Output: 10 -> 20 -> 30 -> null
34 }
35}

```

2. Doubly Linked List

Node Class:

```

java
VerifyOpen In EditorRunCopy code
1public class DoublyNode {
2    int data;
3    DoublyNode next;
4    DoublyNode previous;
5
6    public DoublyNode(int data) {
7        this.data = data;
8        this.next = null;
9        this.previous = null;
10    }
11}

```

Doubly Linked List Class:

```

java
VerifyOpen In EditorRunCopy code
1public class DoublyLinkedList {
2    DoublyNode head;
3
4    // Insert at the end
5    public void insert(int data) {
6        DoublyNode newNode = new DoublyNode(data);
7        if (head == null) {

```

```

8         head = newNode;
9     } else {
10         DoublyNode current = head;
11         while (current.next != null) {
12             current = current.next;
13         }
14         current.next = newNode;
15         newNode.previous = current;
16     }
17 }
18
19 // Display the list
20 public void display() {
21     DoublyNode current = head;
22     while (current != null) {
23         System.out.print(current.data + " <-> ");
24         current = current.next;
25     }
26     System.out.println("null");
27 }
28
29 public static void main(String[] args) {
30     DoublyLinkedList list = new DoublyLinkedList();
31     list.insert(10);
32     list.insert(20);
33     list.insert(30);
34     list.display(); // Output: 10 <-> 20 <-> 30 <-> null
35 }
36}

```

3. Circular Linked List

Node Class:

```

java
VerifyOpen In EditorRunCopy code
1public class CircularNode {
2    int data;
3    CircularNode next;
4
5    public CircularNode(int data) {
6        this.data = data;
7        this.next = null;
8    }
9}

```

Circular Linked List Class:

```

java
VerifyOpen In EditorRunCopy code
1public class CircularLinkedList {
2    CircularNode head;
3
4    // Insert at the end
5    public void insert(int data) {
6        CircularNode newNode = new CircularNode(data);
7        if (head == null) {
8            head = newNode;
9            newNode.next = head; // Point to itself
10        } else {
11            CircularNode current = head;
12            while (current.next != head) {
13                current = current.next;

```

```

14         }
15         current.next = newNode;
16         newNode.next = head; // Make it circular
17     }
18 }
19
20 // Display the list
21 public void display() {
22     if (head == null) return;
23     CircularNode current = head;
24     do {
25         System.out.print(current.data + " -> ");
26         current = current.next;
27     } while (current != head);
28     System.out.println("(back to head)");
29 }
30
31 public static void main(String[] args) {
32     CircularLinkedList list = new CircularLinkedList();
33     list.insert(10);
34     list.insert(20);
35     list.insert(30);
36     list.display(); // Output: 10 -> 20 -> 30 -> (back to head)
37 }
38}

```

Advantages of Linked Lists

- **Dynamic Size:** Can grow and shrink as needed.
- **Efficient Insertions/Deletions:** Adding or removing elements is generally faster than in arrays.
- **No Wasted Space:** Memory is allocated as needed.

Disadvantages of Linked Lists

- **Memory Overhead:** Requires extra memory for pointers.
- **No Random Access:** Must traverse the list to access elements.
- **Cache Performance:** Poorer cache performance compared to arrays.

Applications of Linked Lists in Java

Linked lists are versatile data structures that have numerous applications in programming and software development. Here are some key applications:

1. **Dynamic Memory Allocation:** Allows for dynamic memory allocation, meaning that memory can be allocated as needed during runtime.
2. **Implementation of Other Data Structures:** Used to implement stacks, queues, and graphs.
3. **Real-World Applications:** Used in music players, web browsers, and image viewers for managing lists of items.
4. **Efficient Insertions and Deletions:** Provides $O(1)$ time complexity for insertions and deletions at any position.
5. **Memory Management in Operating Systems:** Can be used for managing processes and scheduling.
6. **Manipulation of Polynomials:** Can represent polynomials where each node contains a coefficient and an exponent.

7. **LRU Cache Implementation:** Can be implemented using a doubly linked list to keep track of the most recently accessed items.

Linked List vs Array Comparison in Java

When comparing **Linked Lists** and **Arrays**, consider the following:

1. **Memory Allocation:** Arrays use contiguous memory; linked lists use non-contiguous memory.
2. **Structure:** Arrays consist of a single data field; linked lists consist of nodes with data and pointers.
3. **Access Time:** Arrays allow $O(1)$ access; linked lists require $O(n)$ traversal.
4. **Insertion and Deletion:** Arrays require $O(n)$ for shifting; linked lists allow $O(1)$ if the position is known.
5. **Memory Efficiency:** Arrays are more efficient when the size is known; linked lists are better for dynamic sizes.
6. **Locality of Reference:** Arrays have better cache performance; linked lists have poorer cache performance.
7. **Use Cases:** Arrays are best for fixed sizes and fast access; linked lists are best for frequent modifications.

Conclusion

Linked lists are a fundamental data structure in Java, providing flexibility and efficiency for dynamic data management. Understanding how to implement and manipulate linked lists is crucial for software development and algorithm design. If you have further questions or need more examples, feel free to ask!